

Chaemin Lim cl5456, Joshua (Kimyung) Song js8604, Ahania Soni as7918

Part I – Literature Review (5 pts)

Pizarro et al. (2022). "Automatic floor plan analysis and recognition". Science Direct.

<https://www.sciencedirect.com/science/article/abs/pii/S0926580522002217>

This review surveys learning-based methods for extracting walls, doors, rooms and other architectural elements from floor plans, focusing on challenges in recovering topology and semantics from rasterized drawings. It is useful for our project because it establishes the hand-drawn sketch → clean room plan pipeline and helps position our work as extending floor-plan recognition into an interactive design workflow.

Khade et al. (2025). "A Comprehensive Survey of Floor Plan Image Analysis and Related Applications". Springer.

<https://link.springer.com/article/10.1007/s10032-025-00528-8>

This review covers a multitude of recent approaches to floor plan image analysis and related applications, providing state-of-the-art overview of the current techniques, datasets, and practical uses. It supports our project by reinforcing the relevance of floor plan image analysis, which will help convert informal image-based room drawings into structured representations.

Xu et al. (2025). "Constraint Is All You Need: Optimization-Based 3D Level Generation with LLMs". Association for Computing Machinery.

<https://dl.acm.org/doi/10.1145/3723498.3723840>

This review proposes a methodology where an LLM translates natural-language text into structured spatial constraints and weights, which are then inputted through algorithmic optimization rather than pre-planned geometric configuration. This is relevant to our project, as the baseline methodology for using an LLM to determine weights for our packing constraints, room orientation, and layout priorities.

Hu et al. (2024). "Raster-to-Graph: Floorplan Recognition via Autoregressive Graph Prediction with an Attention Transformer." *Computer Graphics Forum*.

This paper introduces a framework that converts rasterized floor plan images into structured vector graphs by using a visual attention Transformer to iteratively predict wall junctions and segments. It is relevant to our project's sketch-input feature, as it demonstrates a method for co-recognizing both the geometry and semantics of a floor plan from a raw image in a single network. A key limitation is that the model is trained on professional CAD drawings, meaning performance on rough hand-drawn sketches — like those our users will produce — may require additional adaptation.

Lv et al. (2021). "Residential Floor Plan Recognition and Reconstruction". CVPR.

Residential Floor Plan Recognition and Reconstruction

This paper is useful because it shows that floor-plan drawings can be automatically parsed into structured architectural information such as rooms, walls, and dimensions. For our project, it supports the idea that rough user sketches do not need to stay as raw drawings; they can become inputs to a computational generation pipeline. Its limitation for our purposes is that it is focused more on recognition and reconstruction than on interactive user-guided design.

He et al. (2022). "iPLAN: Interactive and Procedural Layout Planning". CVPR.

[iPLAN: Interactive and Procedural Layout Planning \(CVPR 2022\)](#)

Short review:

This paper is especially relevant because it treats layout generation as an iterative design process rather than a one-shot automated output. It shows how a system can accept user input and gradually refine a layout, which matches our project's focus on interactive procedural generation. Its main value for us is conceptual: it supports designing a tool where the user steers generation instead of only receiving random plans.

arXiv articles not peer-reviewed; referenced for methodology:

Leng et al. (2023). "Tell2Design: A Dataset for Language-Guided Floor Plan Generation".

arXiv:2311.15941.

<https://arxiv.org/abs/2311.15941>

This paper introduces a large dataset of over 80,000 floor plans paired with natural-language instructions, which generates floor plans as a design task with semantic, geometric, and topological constraints. This generation informs our project's LLM pipeline by showing how natural language can express spatial intent, which can be implemented for our packing algorithm.

Qin et al. (2024). "ChatHouseDiffusion: Prompt-Guided Generation and Editing of Floor Plans".

arXiv:2410.11908.

<https://arxiv.org/abs/2410.11908>

This paper combines LLM, topological representation, and diffusion models to support prompt-guided floor plan generation and iterative editing. It is relevant to our project as it demonstrates how language-based layout systems can become more practical through interactive refinement, including our project's sketching and zoning → procedural hotel generation workflow.

Zong et al. (2024). "HouseTune: Two-Stage Floorplan Generation with LLM Assistance."

arXiv:2411.12279.

This paper proposes a two-stage pipeline in which chain-of-thought prompting guides an LLM to generate an initial room layout from natural language, which is then refined by a diffusion model into a geometrically coherent floor plan. It directly informs our text-prompt generation mode, showing that LLMs can bridge natural language and spatial design without large domain-specific datasets. However, the authors note that precise room sizing and placement from text alone remains difficult, suggesting our system should allow users to manually adjust LLM-generated layouts.

Part 2 – Project Definition and Implementation Roadmap (5 pts)

- Project goal: The goal of this project is to build an interactive floor-plan design tool that expands on our midterm random plan generator. Rather than relying only on random procedural packing, users will be able to create plans in multiple ways: by sketching a rough layout, dragging and dropping room zones, or using a random generation mode with adjustable parameters. We also want to explore text input so that natural-language instructions can influence the generated layout.
- Specific objectives: Research on floor plan analysis and recognition supports the idea of translating rough room sketches into cleaner, structured representations that can be stored and reused. More recent work on LLM-guided floor plan generation shows that natural language can encode algorithm weights that influence room types, layout preferences, and overall composition, which is relevant to our exploration of text-based generation. Our project will utilize LLMs for constraint generators instead of direct geometric generators to determine packing logic, while preserving a controllable procedural backend that provides foundational functionalities of random procedural generation and manual editing.
 - Technical objectives:
 - Room authoring workflow where users can either sketch rough room layouts or construct them through drag and drop interactions.
 - Multiple plan generation modes: sketch-based zoning, LLM-guided generation using natural-language constraints, and parameter-controlled or random procedural generation (midterm).
 - Editable generated layouts for user refinement.
- Methodological approach: A modular interactive design system rather than a single generation script will be used. The first “layer” will be room authoring where users can sketch or assemble (drag & drop) reusable room templates. The second layer will be the plan-generation that will expand the midterm’s procedural packing logic to support multiple forms of input, including zoning constraints and adjustable generation parameters. If implemented, the natural-language component will likely use an LLM API to translate prompts into weights for layout constraints (room counts, orientation preferences, or spatial priorities). The third layer will be an editing interface that allows users to adjust generated outputs.
- Implementation roadmap (in order):
 - Clean up and modularize current midterm generator and improve UI controls

- Develop room authoring interface, starting with drag & drop and then sketch-based room creation.
- Zoning-based generation mode where users can define general regions for different room types.
- Prompt-based generation via LLM for constraints and algorithm weights that influence packing algorithms.
- Usability improvements, plan editing, contextual visualization.
- Success criteria: Success will be evaluated on technical functionality and interaction quality. At a minimum, the system should allow users to define or store room templates and generate hotel plans through multiple interaction modes. Generated plans should be spatially coherent, non-overlapping, and understandable as hotel layouts, while allowing user edits and regeneration. If natural-language input is implemented, there should be visible effects on the generated composition rather than arbitrary or opaque changes. Overall, the project will be successful if it demonstrates a coherent workflow that expands the midterm project into a more interactive, mixed-initiative design tool.

Part 3 – System Design, UI Strategy, and Technical Implementation Plan (5 pts) Define the functional architecture and interaction model of your application. Include:

- Application functionalities: The proposed application is an interactive floor-plan design tool that expands our midterm random procedural generator into a more flexible and human-computer-interaction HCI-centered system. Users will be able to create room plans by sketching rough layouts or assembling them with drag-and-drop interactions, then store those rooms as reusable templates. At the site scale, the application will support multiple generation modes: hand-drawn zoning, random procedural generation with adjustable parameters (midterm), and a possible text-input mode where natural-language instructions influence layout behavior. The system will output editable hotel plans with site metrics, features, and representations that will help curate spatial composition.
- User interaction and UI design: UI is designed as a mixed-initiative workflow where users can move between drawing, selecting, generating, and editing rather than a black-box generator. Interface would likely begin with room authoring canvas, alongside a small library panel for saving reusable room templates. The next workshape would focus on the site generation with multiple options between zoning, random, or language-guided modes. The overall interface would have sliders and toggles that allows users to adjust site size, room counts, spacing, room types, etc, which will highlight transparency and iterative control rather than full automation.
- System architecture and data flow: System architecture would be modular so that each generation mode can share common room and layout data rather than requiring repetitive yet separate pipelines. Input layer collects user actions such as sketch geometry, drag-and-drop orders, zoning regions, adjustable parameters, natural-language prompts, etc. The processing layer translates these inputs into room templates, site constraints and weights, room counts, generation rules, etc, and allows

generation via zoning, prompt-based, or procedural packing. The editing layer allows users to make any final polishing decisions by editing selected parts of the plan.

- Implementation plan: Implementation would build on the modular python structure as developed in the midterm projects. Files such as [config.py](#), [room.py](#), [packing.py](#) would be expanded, while additional modules will be introduced for zoning logic, interaction handling, potential natural-language based weight determination, etc. The packing algorithm would be specifically refactored so it can accept different input sources instead of the simple random generation algorithm. Specifically, a dedicated file will convert user prompts into structured constraints for room preferences, adjacency goals, orientation, weighting adjustments, etc. An interaction module will manage sketch-to-room-template, drag and drop actions, and any other interface events.

Part 5 – Testing and Validation (Unit Testing) (5 pts)

`test_suite.py`

We focused our tests on the parts of the system that everything else depends on — the geometry helpers, hotel model, and packing algorithm. Using Python's unittest framework, we checked normal behavior, edge cases, and bad inputs for each module. The packing algorithm was the hardest to test because it uses randomness; we handled this by fixing seeds so results are consistent, then checking spatial rules like no overlapping rooms and all rooms staying within the site boundary rather than checking exact outputs. The room filter modes were verified by confirming that generated rooms only included the expected types. Testing the LLM bridge focused on key detection, fallback behavior when the API fails, and weight parsing. One thing we learned is that UI and rendering code is genuinely difficult to unit test without running the full application — separating the data model from the display layer more cleanly would make future testing much easier.

Part 6 - User Evaluation and Iterative Improvement

We conducted informal user testing with three participants. Each participant began with a different focus mode, random, zoning, or LLM, but all three were asked to move through the full InterPlan workflow: room authoring, generation, switching between modes, reviewing outputs, and final refinement.

- User 1: Random Generation Focus

The first user began with random generation by increasing bedroom bias, lowering padding, and producing a dense hotel layout with many private rooms and limited green space. This confirmed that random generation works well for fast exploration and high occupancy layouts. After this, the user also tested room authoring by selecting preferred bedroom templates, then moved into zoning and LLM prompts to see whether similar density goals could be controlled more intentionally.

Their main feedback was that InterPlan was strongest when random generation acted as a starting point rather than a final answer. They liked quickly producing many options, then using

other modes to refine intent. The main issue was that some controls, especially padding, bedroom bias, and bush density, were unclear at first. We responded by clarifying slider labels and making the metrics panel more useful for judging density, built area, open area, and room type counts.

- User 2: Zoning Generation Focus

The second user began with zoning by drawing a long bedroom zone like a cabin layout, then placing a library as a central public anchor. This showed that zoning makes spatial organization explicit while still allowing variation within each region. After testing zoning, the user moved back into room authoring to adjust available room types, then compared the result with random generation and LLM generated zones.

Their strongest feedback was that zoning felt the most architectural because it let them define program relationships before generation. However, they initially expected zones to behave like exact containers. When a zone was too small, the system could not always fit every requested room because it still had to respect room size, padding, and overlap constraints. We improved zone labels and added clearer feedback when a zone was too restrictive. A future version could show estimated zone capacity before generation.

- User 3: LLM Generation Focus

The third user began with LLM generation through prompts like “20 rooms,” “12 rooms bedroom only,” and “6 rooms public rooms only.” They then tested spatial prompts, asking for bedrooms in the northeast corner and greenery in the southwest. This showed that the LLM could translate language into structured constraints, including room count, room type, zone location, and landscape intent. Afterward, the user adjusted the results through zoning and room authoring rather than treating the LLM output as final.

Their main feedback was that LLM generation made InterPlan approachable because they could start with plain language instead of learning every control first. However, short prompts sometimes felt ambiguous, and the user wanted to know how the app interpreted their instructions. In response, we added an interpretation summary before generation, showing detected room count, room type, zone, and landscape intent.

Overall, the testing showed that our project works best as an iterative workflow, not three separate generators. Random generation was strongest for speed and variation, zoning was strongest for spatial control, and LLM generation was strongest for accessibility. Across all users, the biggest challenge was transparency: users wanted to understand how their inputs became constraints. Future improvements include clearer constraint previews, zone capacity estimates, seed history, room locking, and selective regeneration so users can preserve strong parts of a plan while refining weaker areas.

Part 8 – Project Summary and Critical Reflection

This project addresses the challenge of creating flexible and intuitive tools for generating architectural floor plans that balance algorithmic efficiency with user control. Traditional procedural systems often produce valid layouts but lack meaningful user input, while manual design tools require significant time, effort, and expertise. Our application bridges this gap by enabling users to guide layout generation from the beginning to the very end through interactive inputs while maintaining computational structure.

The core workflow is organized into three stages: input, processing, and generation. Users begin by defining room layouts through drag-and-drop authoring with provided room components (i.e. furniture, bathrooms, etc) or broadly pre-set room templates, and specifying adjustable parameters like room count and spacing. Users can further define plans by zoning regions, interacting with an LLM to handle the plan composition, or simply running a randomization algorithm. These inputs are translated into structured weight constraints that influence the procedural packing algorithm, which generates non-overlapping layouts within a defined site. The system then outputs an editable floor plan along with computed metrics such as density, built area, and room distribution. Users can iteratively refine and regenerate layouts, supporting a mixed-initiative design process.

A key strength of the implementation is its ability to unify previously separate architectural workflows into a single integrated process. It brings together manual design input through drag-and-drop authoring, procedural generation through algorithmic packing, and emerging AI-driven approaches for constraint-based guidance. By connecting these traditionally disconnected systems, the application allows users to move seamlessly between designing, generating, and refining layouts within one environment, improving both efficiency and creative control. However, the current implementation has limitations, including a relatively simple user interface, limited constraint expressiveness, and incomplete integration of advanced inputs such as natural language guidance.

Future improvements could include a more robust graphical UI, expanded constraint systems for adjacency and circulation, and deeper integration of LLM-based inputs to better translate user intent into spatial and packing logic. Additional enhancements such as performance optimization, richer room datasets, and improved editing capabilities would further strengthen the system's usability and design potential.

Part 9 - Screen Recording

- In the zip folder as mp4.

Part 10 - Final Review Presentation

[Link to Presentation](#)